

Costo in gate del circuito di Trotter

Note di lavoro — Tesi triennale, Samuele Schiavi
Relatore: Prof. Alessandro Chiesa

Analisi del numero di gate del circuito per il singolo passo di Trotter e per N passi, con la decomposizione `cx-rz-cx` al posto del gate nativo `RZZ` (vedi la sostituzione applicata a `quantum_simulation_dimero_trotter_semplificato.ipynb` e `quantum_simulation_dimero_trotter.ipynb`).

1 Per un singolo passo di Trotter

Con la decomposizione attuale (`cx-rz-cx` al posto di `RZZ`):

Tipo	Quantità
<code>rz</code> (campo + le 3 rotazioni sostitutive di <code>RZZ</code>)	5
<code>h</code> (Hadamard, coniugazione del blocco DM)	4
<code>rx</code> , <code>ry</code>	1 + 1
<code>cx</code> (3 coppie CNOT–Rz–CNOT)	6
<code>barrier</code> (separatore, non un gate fisico)	1

Totale: 17 gate per passo (9 a un qubit + 8 a due qubit: $1 R_{XX} + 1 R_{YY} + 6 CX$), profondità 15.

Scala linearmente con N

gate totali = $17N$, gate a 2 qubit = $8N$, profondità = $15N$

N	gate 1-qubit	gate 2-qubit	totale	profondità
1	9	8	17	15
2	18	16	34	30
5	45	40	85	75
20	180	160	340	300
50	450	400	850	750
100	900	800	1700	1500

La riga $N = 20$ è quella riportata nel notebook (§8, esecuzione con shot a $t = 30$): **160 gate a due qubit**, coerente col numero che si vede nel notebook.

2 Alternativa più efficiente: gate nativo `RZZ`

La decomposizione `cx-rz-cx` non è l'unica scelta possibile: Qiskit mette a disposizione anche il gate primitivo `RZZGate`, richiamabile con `qc.rzz(theta, 0, 1)`, che implementa la stessa operazione unitaria $R_{ZZ}(\theta) = e^{-i\frac{\theta}{2}ZZ}$ in un'unica istruzione **astratta**, senza scomporla esplicitamente in gate a un qubit e CNOT.

Da dove viene: la `.definition` di Qiskit

`RZZGate` non è un gate "magico" indipendente: porta con sé una `.definition` interna, che Qiskit costruisce automaticamente ed è **esattamente** la stessa decomposizione `cx-rz-cx` usata sopra:

Circuito (per $\theta = 0.5$, da `RZZGate(0.5).definition`)

```
q_0: ----[X]-----[X]----
          |           |
q_1: -----*-----[Rz(0.5)]-*----
```

Questo conferma che le due versioni del circuito sono **identiche a livello di operazione fisica**: cambia solo il livello di astrazione a cui viene scritto e contato il circuito, non il calcolo che rappresentano.

- Scrivendo `qc.rzz(theta, 0, 1)`, Qiskit tiene l'istruzione **come blocco unico** finché non la si scompone esplicitamente (`qc.decompose()`) o si passa per `transpile()`.
- Solo al `transpile()` verso un *basis gate set* che non include `rzz` nativamente (es. `{cx, rz, sx, x}`, tipico di molti backend a superconduttori IBM), il transpiler la espande automaticamente nella stessa sequenza `cx-rz-cx` — cioè fa lui, in automatico, il lavoro che nella versione "esplicita" del notebook viene fatto a mano nel codice Python.

Perché è più efficiente (nel conteggio astratto)

Contando le operazioni **prima** di qualunque `transpile`, un gate `rzz` conta come **1** operazione, non 3 (`cx+rz+cx`). Il risparmio per passo è quindi di $3 - 1 = 2$ operazioni per ciascuna delle tre RZZ presenti nel passo (una in H_1 , due in H_2), cioè **6 operazioni in meno per passo**.

Per un singolo passo di Trotter (versione RZZ nativa)

Tipo	Quantità
<code>rz</code> (solo il campo, 2 rotazioni)	2
<code>h</code> (Hadamard, coniugazione del blocco DM)	4
<code>rx</code> , <code>ry</code>	1 + 1
<code>rzz</code> (le tre esponenziali ZZ , come blocco unico)	3
<code>barrier</code>	1

Totale: 11 gate per passo (6 a un qubit + 5 a due qubit: $1 R_{XX} + 1 R_{YY} + 3 R_{ZZ}$), profondità 9.

Scala linearmente con N

gate totali = $11N$, gate a 2 qubit = $5N$, profondità = $9N$

N	gate 1-qubit	gate 2-qubit	totale	profondità
1	6	5	11	9
2	12	10	22	18
5	30	25	55	45
20	120	100	220	180
50	300	250	550	450
100	600	500	1100	900

Confronto diretto fra le due versioni

Grandezza (per passo)	cx-rz-cx esplicito	RZZ nativo	differenza
gate a 1 qubit	9	6	−3
gate a 2 qubit	8	5	−3
totale	17	11	−6
profondità	15	9	−6

A $N = 20$ ($t = 30$, come nel notebook): 340 vs 220 gate totali, 160 vs 100 gate a due qubit, profondità 300 vs 180.

Quando conviene l'una o l'altra

- **RZZ nativo (astratto)** è la scelta più efficiente ed è quella corretta se l'obiettivo è ragionare sul circuito a livello di algoritmo, o se il backend di destinazione supporta **RZZ** come gate fisico nativo (es. alcuni processori a ioni intrappolati o superconduttori con accoppiamento ZZ diretto): in quel caso il conteggio astratto coincide con quello reale su hardware, e usare la forma esplicita **cx-rz-cx** aggiungerebbe solo gate superflui che il transpiler dovrebbe poi ri-ottimizzare.
- **cx-rz-cx esplicito** è la scelta più realistica quando il backend di destinazione ha **solo CNOT** come gate nativo a due qubit (il caso più comune sui processori a superconduttori IBM): in quel caso il conteggio astratto di **RZZ** sottostimerebbe il costo reale, perché il transpiler dovrà comunque espanderla in **cx-rz-cx** più avanti — scriverla esplicita nel notebook rende visibile fin da subito il costo che il circuito avrà davvero una volta compilato su quel tipo di hardware.

In sintesi

Le due versioni non sono in competizione come "corretta vs sbagliata", ma rappresentano due livelli di astrazione diversi della stessa identità matematica; quale usare dipende da cosa si vuole misurare (costo algoritmico astratto, o costo di compilazione su un hardware con un set di gate nativi specifico).